

Développement des systèmes Communicants

Présentation générale du cours

Ce cours n'est pas un cours de réseau

- On y détaillera pas de protocoles de bas niveau (mais on en parlera)

Ce cours est un cours de programmation réseau

- On y apprend à écrire des applications nécessitant de la communication

Pour le programmeur, qu'est-ce qu'un réseau ?

- Un ensemble de **services**
- Un **moyen** de communication (ensemble de machines/applications atteignables)

Contenu du cours

Partie Théorique

Chapitre 1

- Introduction sur les Prog Réseaux et le protocole HTTP

Chapitre 2

- Fonctionnement des Prog. Réseau: le protocole http

Chapitre 3

- Gestion de flux en java

Chapitre 4

- Les Sockets dans java

Partie Pratique

- Lancement de Mini-Projets

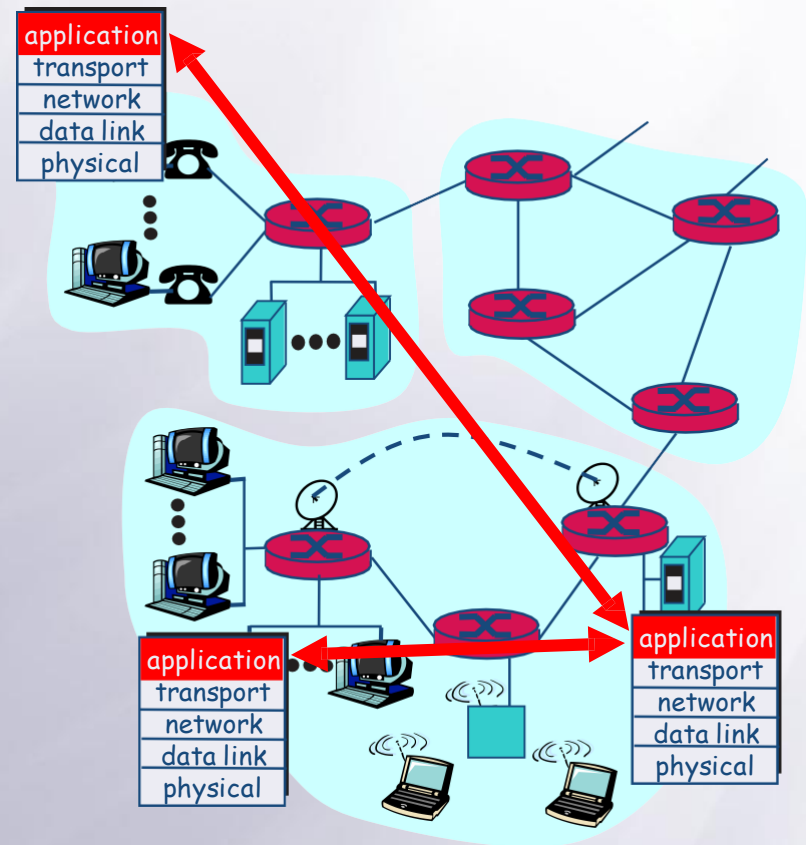


Introduction sur les Prog. réseau et le protocole HTTP



Couche application

- Applications réseau = raisons d'être des réseaux informatiques
- Nombreuses applications créées depuis 30 ans
 - Text-based (80s) : accès distant, email, transfert de fichiers, newsgroups, forums de discussion
 - Multimédia : WWW, téléphonie Internet, vidéoconférence, audio et vidéo à la demande
- Souvent : logiciels distribués entre plusieurs systèmes
 - Communication entre les applications



Applications réseaux : le jargon

 Les entités communicantes sont des processus

 Un processus est un programme qui s'exécute sur un hôte

 Deux processus communiquent dans un même hôte avec des communications interprocessus définies par le système d'exploitation

 Deux processus s'exécutant sur deux hôtes différents communiquent en s'échangeant des messages avec un protocole de couche application

 Processus émetteur / récepteur

 Le processus suppose qu'il existe une infrastructure de transport en-dessous

Applications

- S'exécutent dans les hôtes dans l'espace utilisateur
- Échangent des messages
- Ex : email, transfert de fichier, Web

Protocoles applicatifs

- Une (grosse) partie des applications
 - Web : standards pour les formats de documents (HTML), browsers Web, Prog.réseau et le protocole applicatif HTTP pour définir l'échange des messages
 - Courrier électronique : serveurs de mails (qui hébergent les boîtes aux lettres), mail readers, standard pour le format des mails et protocole applicatif pour l'échange des messages (ex : SMTP)
- Définissent :
 - le format et l'ordre des messages échangés par les applications,
 - les actions consécutives à l'émission ou la réception d'un message
- Certains services sont proposés par les protocoles de couches inférieures (segmentation, contrôle d'erreurs..)

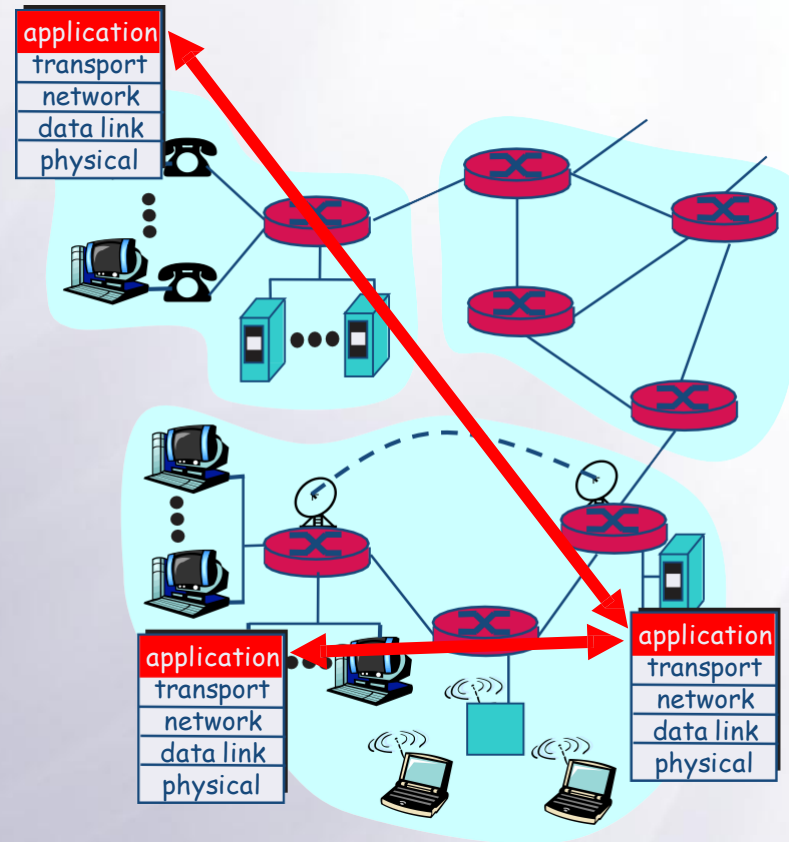
Protocole applicatif

Définit :

- Le type des messages échangés : requête, réponse...
- La syntaxe des différents types de messages : champs du message
- La sémantique des champs, c'est-à-dire la signification des informations qui y sont contenues
- Les règles pour déterminer quand et comment un processus envoie des messages et y répond

Certains protocoles applicatifs sont spécifiés dans des RFC : domaine public (ex HTTP)

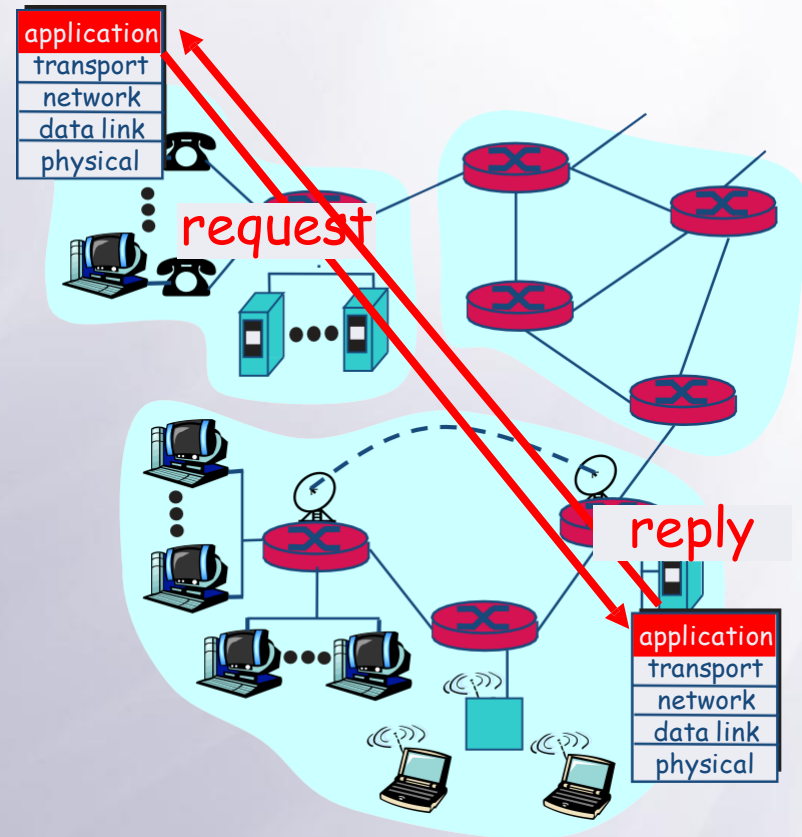
Beaucoup sont propriétaires (ex téléphonie IP)



Paradigme client-serveur

Les réseaux typiques ont deux parties : le **client** et le **serveur**

- Le côté client d'un système terminal communique avec le côté serveur d'un autre système
 - Web : Un navigateur Web implémente le côté client de HTTP, le serveur Web : le côté serveur ;
 - Email : le serveur de mail émetteur implémente le côté client de SMTP, le serveur de mail récepteur : le côté serveur de POP3 ou IMPAP.



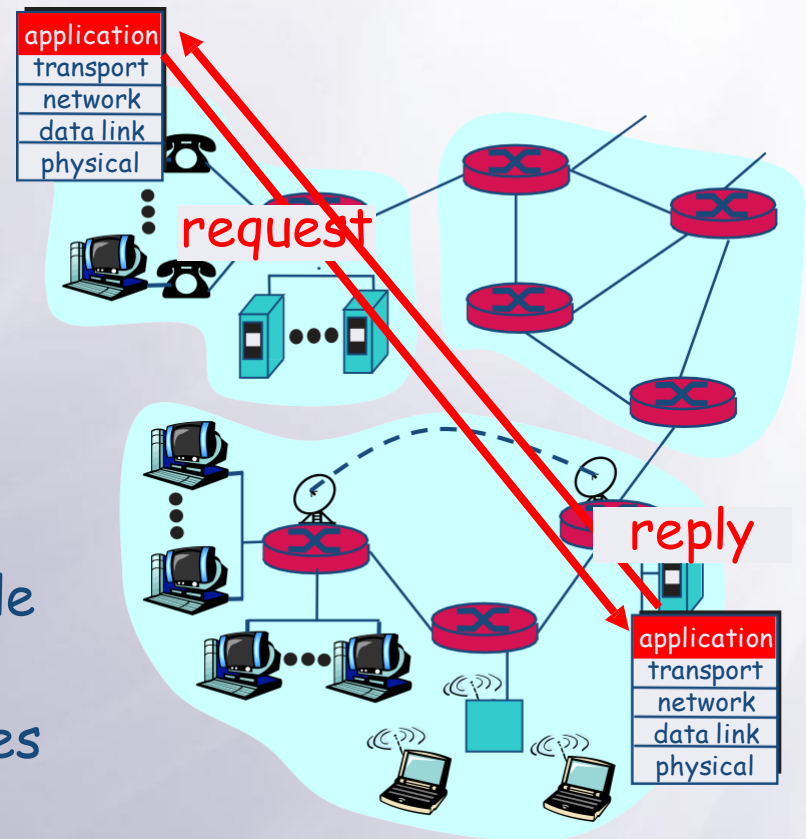
Paradigme client-serveur

Client :

- Initie le contact avec le serveur (il parle en premier)
- Typiquement, il demande un service au serveur
- Pour le Web, le client est implanté dans le browser ;
Pour l'e-mail, dans le mail reader

Serveur :

- Propose les services demandés par le client
- Ex : Le serveur Web envoie les pages Web demandées
- **NB** : Une même machine peut implémenter les côtés client ET serveur



API : Application Programming Interface

- ☰ Définit l'interface entre l'application et la couche transport

- ☰ Socket : API Internet

 - Deux processus communiquent en émettant et recevant des données via les sockets

La connexion réseau

- Le principe général de l'établissement d'une connexion réseau est que:
 - L'un des deux points de communication $[m,p]$ doit être placé en attente d'une demande de connexion (mode serveur)
 - L'autre point de communication $[m',p']$ peut alors demander l'établissement d'une connexion (mode client)
 - la partie serveur décide d'accepter ou non la connexion

et si oui

- Un nouveau point de communication $[m, p'']$ est construit (à l'aide d'un port éphémère) et connecté avec le point de connexion du client
- Une connexion est alors établie entre $[m', p']$ et $[m, p'']$
 - à ce moment les deux points sont fonctionnellement interchangeables (symétrie de la liaison)

La connexion réseau

☰ Une communication standard s'établit de la façon suivante :

- la partie serveur est passive et attend toute demande de connexion sur un port donné
- la partie cliente choisit un port depuis lequel elle va émettre sa demande de connexion
- le serveur accepte la demande en établissant la connexion sur un port de service
- lorsque tout est établi, on aura bien un couple identifiant la connexion...

La connexion réseau: la socket

☰ L'objet standard de communication est la **socket**

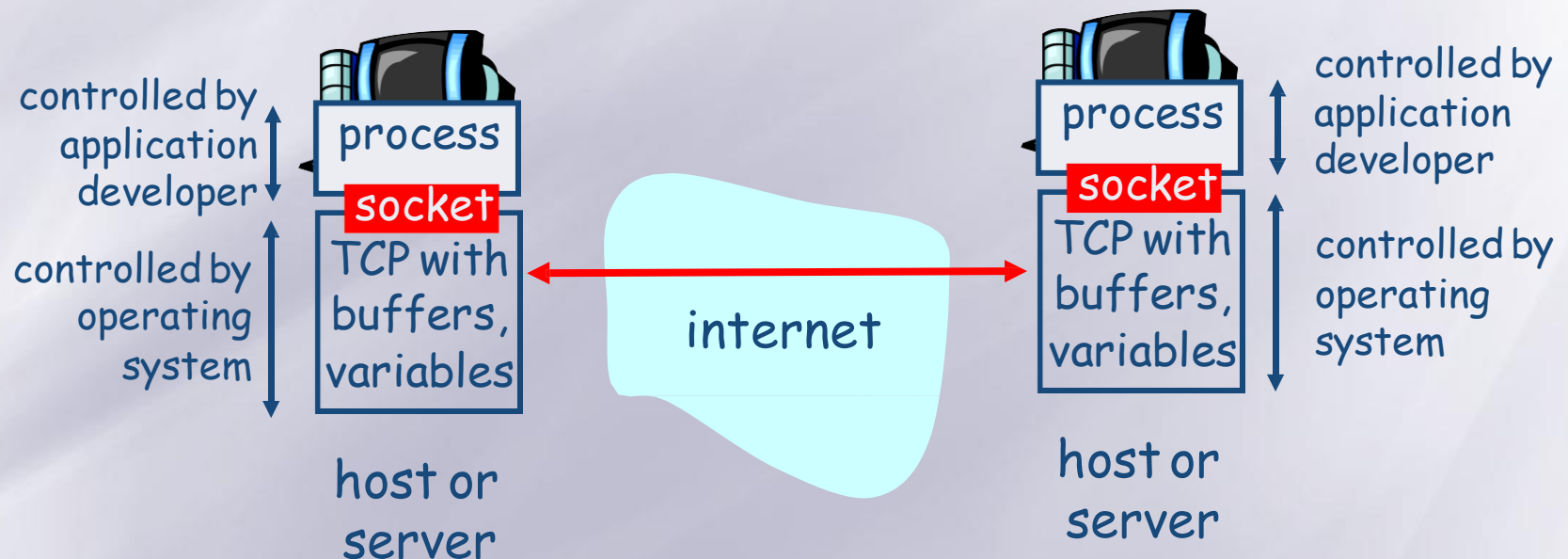
- de quoi s'agit-il ?
 - il faut avoir à l'esprit que dans l'univers inspiré d'Unix « tout est fichier »
 - un abus de langage pour signifier l'uniformité d'opérations d'entrées/sorties : « tout se manipule de la même manière en matière d'entrées/sorties »
 - y compris le réseau... où l'objet sous-jacent correspondant est appelé **socket**



☰ Les **sockets** sont des flux de données, permettant à des machines locales ou distantes de communiquer entre elles.

Socket

- Porte d'entrée d'un processus
- Interface entre la couche application et la couche transport d'un hôte
- Le développeur contrôle toute la partie application des sockets ; il n'a que peu de contrôle sur la partie transport (choix du protocole et éventuellement ajustement de quelques paramètres)
- Socket = API Internet



Introduction: Adressage et Point de Communication

- ❑ La couche transport fournit un ou plusieurs services de communication entre applications (par exemple TCP ou UDP)
 - ▣ Pour communiquer, ces applications doivent se connaître
 - Il existe donc un mécanisme de **nommage**...
- ❑ Une application fournit un service particulier sur une machine donnée
 - ▣ Le service est identifié par un **port**
 - ▣ La machine est identifiée par une **adresse**
 - Ces informations sont similaires à celles jouées par un numéro de bureau (port) dans une administration donnée (machine)

Introduction: Adressage et Point de Communication

□ Un couple (adresse, port) est un **point de communication**

▣ toute **communication** ne peut s'effectuer qu'entre au moins **deux points** de communications

▣ Pour plus

■ l'émetteur

■ le (ou les) **receveur(s)**

Introduction: Adressage et Point de Communication

- Une machine peut-être identifiée par :
 - ▣ un **nom Internet** (pas strictement nécessaire)
 - par exemple : www.samsung.com
 - une machine peut posséder plusieurs noms
 - ▣ par une **adresse Internet** (toute machine connectée au réseau en possède une)
 - en réalité, il s'agit de l'adresse d'un dispositif réseau sur une machine (ex : 211.45.27.202)
 - une adresse par dispositif (carte wifi, carte Ethernet)
 - mais possiblement plusieurs dispositifs pour une machine

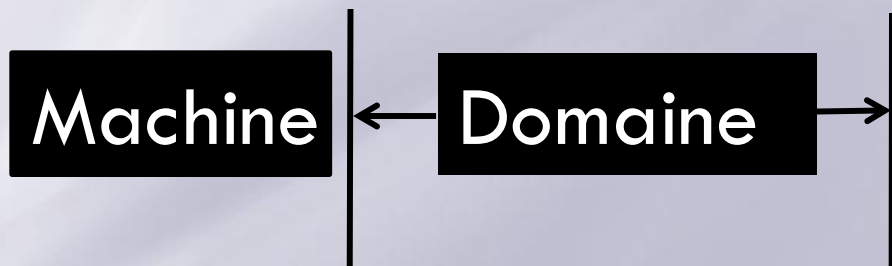
Structure de l'adresse internet

- ❑ Les adresses correspondent à une organisation structurelle du réseau
 - ❑ Les machines préfèrent les nombres
 - ❑ Les noms correspondent à une organisation logique
 - ❑ Les humains préfèrent les noms
 - ❑ Pensez à la situation des liens et des inœuds dans les systèmes de fichiers de la famille Unix:
 - inœud → structure disque
 - lien → structure logique
- Exemple: Annuaire téléphonique

Structure de l'adresse internet

- ❑ Le nom entièrement qualifié d'une machine est une représentation hiérarchique de la structure logique à laquelle elle appartient: On identifie une machine dans un domaine.
- ❑ Le nom est constitué de deux parties

www.institut.rnu.tn



La spécification du domaine représente la hiérarchie des responsabilités

exemple : L'**institut** fait parti du réseau national universitaire est située dans le domaine tunisien.

le domaine le plus à droite est appelé domaine de **premier niveau** (top-level domain)

- en gros, deux types :

génériques (ex. : .com, .edu, ...)

nationaux (ex. : .fr, .tn, .uk, .de, .it...)



pour www.informatique.univ-paris-diderot.fr

- fr est le domaine national attribué par l'ICANN à la france avec délégation à l'AFNIC
- univ-paris-diderot est le sous-domaine attribué par l'AFNIC à l'université paris diderot avec délégation à la DSI de l'université
- informatique est le sous-domaine attribué par la DSI à l'UFR avec délégation au service informatique de l'UFR
- www est le nom d'une des machines sous la responsabilité de l'UFR

☰ Le service permettant de faire la translation d'un nom en une adresse s'appelle le service de nom, c'est un *annuaire*

☰ le système aujourd'hui le plus répandu est le DNS (RFC 882) qui est un service d'annuaire distribué.

Important : les communications nécessitent la connaissance des adresses Internet des machines concernées

Les types d'adresses internet

☰ On notera au passage qu'il existe aujourd'hui deux types d'adresses : IPv4 et IPv6(respectivement 4 octets et 8 groupes de 2 octets)

☰ La notation décimale employée pour les adresses IPv4 (par exemple 172.31.128.1) est abandonnée au profit d'une écriture hexadécimale, où on a 8 groupes de 2 octets (soit 16 bits par groupe) sont séparés par un signe deux-points :

- Exemple 2001:0db8:0000:85a3:0000:0000:ac1f:8001
- Deux réseaux qui cohabitent...

☰ Sur une machine donnée plusieurs communications peuvent prendre place simultanément

- Par conséquent, il doit être possible de les distinguer, comme pour différencier les téléphones fixes dans une entreprise, il existe la notion de **port**

- ✓ toute communication nécessite l'utilisation d'un port
- ✓ les ports sont donc utilisés pour assurer le **multiplexage**, un flux est identifié par deux couples (adresse,port)

☰ Les ports UDP et TCP sont distincts (deux espaces)

☰ Il existe essentiellement trois types de ports :

- **Les ports reconnus:** numéros compris entre 0 et 1023
- **Les ports réservés:** numéros compris entre 1024 et 49151
- **Les ports libres:** numéros compris entre 49152 et 65535

☰ Les ports reconnus (Well-known ports) sont utilisés par des services réseau d'usage général et commun :

- 20 et 21 pour FTP
- 25 pour SMTP
- 80 pour HTTP

☰ Ce qui signifie que pour établir une connexion avec un serveur web, il faut s'adresser au port 80 de la machine concernée

☐ Les ports réservés (Registered ports) :

- certains correspondent à des services d'usage moins général (souvent des services propriétaires)

- ☐ 17500 pour Dropbox

- n'importe quelle application peut en faire l'usage quelle désire...

☐ Les ports libres (Dynamic, private or ephemeral ports)

- normalement utilisés pour des durées limitées...

Protocole applicatif

Adressage des processus

Comment un processus identifie-t-il un processus distant pour communiquer ?

- **Nom ou adresse de l'hôte distant :**

- **adresse IP** de l'hôte distant : 32 bits qui identifient de manière unique l'interface qui connecte l'hôte à l'Internet

- **Identifiant du processus récepteur chez l'hôte distant**

- **“Numéro de port”** : permet de différencier les différents processus locaux auxquels le message doit être transmis
- Les protocoles applicatifs usuels ont des numéros de port réservés :
 - 80 pour le processus serveur Web
 - 25 pour le protocole de serveur de mail (utilisant SMTP)
 - Well-know port numbers : RFC 1700
 - Un nouveau numéro de port est affecté à chaque nouvelle application



Agent utilisateur

- Un agent utilisateur est une interface entre l'utilisateur et l'application réseau
 - Web : browser
 - Visualisation de pages Web
 - Navigation sur le Web
 - Interactions avec les applets Java
 - Implémentation du côté client du protocole HTTP
 - Interface avec l'utilisateur : processus qui envoie et reçoit les messages via une socket
 - E-mail : eudora, outlook
 - streaming audio/vidéo : real player, media player



Modes de Communication

Il existe au moins deux grands modes de communication :

- par **paquet** (datagram) : la poste en est un bon exemple
 - ✓ dans ce mode on n'est pas connecté, on est juste atteignable (ayant une adresse)
- en **flux** (stream) : la téléphonie en est un bon exemple
 - ✓ dans ce mode on est connecté (on exige la présence de l'autre pour décrocher)

Mode par paquet

☰ dans le mode paquet :

- Il n'existe pas d'ordre dans la délivrance des paquets
 - ✓ Il n'existe pas non plus de fiabilité
- Un paquet posté en premier peut arriver en dernier
 - ✓ un paquet envoyé peut être perdu

☰ **Avantage : souple et léger...**



Mode par Flux

☰ dans le mode flux :

- les informations sont reçues dans l'ordre exact de leur émission
- il n'y a pas de perte

☰ **inconvénient** : nécessite l'établissement d'une connexion et consomme des ressources



Mode par paquet

☰ Comment ceci fonctionne-t-il dans la réalité ?

- Le service de base est un mode paquet non fiable
 - ✓ la perte peut être compensée par des réémissions
 - ✓ la dégradation peut être compensée par des codages

☰ Comment on peut passer d'un tel service à des services fiables ?

- En superposant des *couches logicielles* agrémentant les couches inférieures de propriétés attendues (ajout des mécanisme ex: date, num de paquet pour rétablir l'ordre))

Quel est le service de transport nécessaire à une application?

 **Socket = interface entre le processus applicatif et le protocole de transport**

- Côté émetteur : l'application envoie des messages par la porte
- De l'autre côté de la porte, le protocole de transport doit déplacer les messages à travers le réseau, jusqu'à la porte du processus récepteur

 **De nombreux réseaux (dont Internet) fournissent plusieurs protocoles de transport**

- Lequel choisir lorsqu'on développe une application ?
 - Étude des services fournis par chaque protocole
 - Sélection du protocole qui correspond le mieux aux besoins de l'application

Quel est le service de transport nécessaire à une application?

 **3 types de besoins au niveau des applications, en termes de :**

- **Fiabilité (influence du taux de perte de données)**
- **Bande passante**
- **Délai**



Quel est le service de transport nécessaire à une application?

Fiabilité : Perte de données

☰ Certaines applications nécessitent une fiabilité à 100%

- Courrier électronique (SMTP)
- Transfert de fichiers (FTP)
- Accès distant (Telnet)
- Transfert de documents Web (HTTP)
- Applications financières

☰ D'autres peuvent tolérer des pertes (loss-tolerant applications)

- Applications multimédia : audio/vidéo

Quel est le service de transport nécessaire à une application?

Bande passante

- Certaines applications (ex : multimédia) requièrent une bande passante minimale
 - Téléphonie sur Internet : si la voix est codée à 32 Kbps, les données doivent être transmises à ce débit
 - Applications multimédia
- D'autres utilisent la bande passante disponible (applications élastiques)
 - Courrier électronique, transfert de fichiers, accès distant, Web
 - Plus il y a de bande passante, mieux c'est !
 - One cannot be too rich, too thin or have too much bandwidth

Quel est le service de transport nécessaire à une application?

Délai

- ☐ Certaines applications nécessitent un délai de bout-en-bout faible (moins de quelques centaines de ms)

- Applications temps réel interactives :

- Téléphonie sur Internet
- Environnements virtuels
- Téléconférence
- Jeux en réseau

- ☐ Pour les application non temps réel, un délai court est préférable, mais pas de contrainte forte

Besoin en service de transport

Application	Pertes	Bande passante	Sensibilité temp.
Transfert de fichiers	Sans pertes	élastique	Non
e-mail	Sans pertes	élastique	Non
Web	tolérant	élastique	Non
Audio/vidéo Temps réel	tolérant	audio: 5Kb - 1Mb vidéo: 10Kb - 5Mb	Oui, 100's ms
Audio/vidéo enregistré	tolérant	idem	Oui, quelques s
Jeux interactifs	tolérant	Quelques Kbps	Oui, 100's ms
Applis financières	Sans pertes	élastique	Oui et non

Services proposés dans Internet

Service TCP:

- **Orienté connexion:** connexion nécessaire entre le client et le serveur
- **Transport fiable** entre le processus émetteur et récepteur
- **Contrôle de flot:** l'émetteur ne submerge pas le récepteur
- **Contrôle de Congestion :** réduit le débit de l'émetteur quand le réseau est congestionné
- **Ne propose pas:**
 - de garanties de délai,
 - de bande passante minimale

Service UDP:

- **Transfert de données non fiable**
- **Ne propose pas**
 - de connexion,
 - de fiabilité,
 - de contrôle de flot,
 - de contrôle de congestion,
 - de garantie temporelle,
 - de bande passante

Fonctionnement des Prog. réseau : le protocole HTTP



Pourquoi on doit connaître http ?

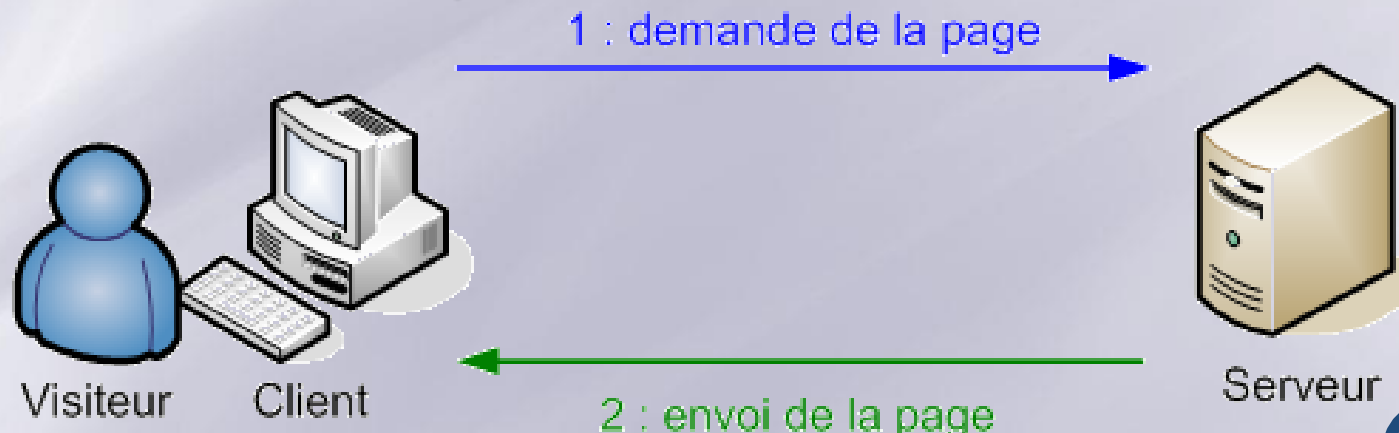
Le protocole http est tellement omniprésent sur l'Internet: une connaissance minimale de l'HTTP et du vocabulaire qui l'entoure est bien souvent très utile.

Par exemple cela pourra nous permettre :

- ☐ de savoir de quoi on parle quand on entend les mots GET ou POST
- ☐ de savoir comment gérer au mieux une redirection d'Url
- ☐ de savoir ce qui se cache derrière un message "404 File not found" affiché par un navigateur
- ☐ pour vérifier des points techniques (compression, erreurs...) et le SEO de son site (les redirections 301, 302...)

http: HyperText Transfer Protocol

- ❑ **HTTP**, littéralement le « protocole de transfert hypertexte », est un protocole de communication client-serveur développé pour le World Wide Web.
- ❑ Il est utilisé pour échanger toute sorte de données entre client HTTP et serveur HTTP.
- ❑ HTTP est un protocole qui va permettre au client (par le biais d'un navigateur par exemple) de communiquer avec un serveur connecté au réseau (le serveur HTTP installé sur le serveur d'un site, par exemple *Apache*).



Le Web : jargon

Page Web :

- Contient des “objets”
- Adressée par une URL

La plupart des pages Web contiennent :

- Pages HTML de base
- Objets référencés

L'URL a deux composants :

- nom d'hôte
- chemin d'accès

www.someSchool.edu/someDept/pic.gif

L'Agent Utilisateur pour le Web est le browser :

- MS Internet Explorer
- Netscape Communicator

Le serveur Web :

- Apache (domaine public)
- MS Internet Information Server

Le Web : le protocole HTTP

HTTP : HyperText Transfer Protocol

■ Couche applicative Web

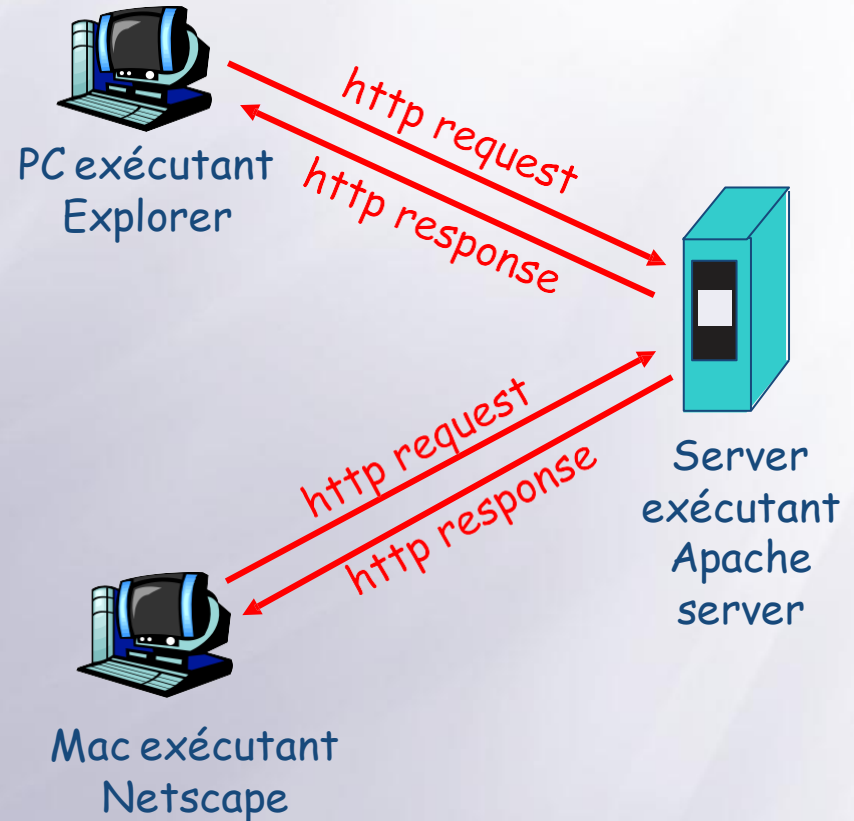
■ Modèle client/serveur

● **Client** : le browser, qui demande, reçoit, affiche les objets Web

● **Serveur** : le serveur Web, qui envoie les réponses aux requêtes

■ http1.0 : RFC 1945

■ http1.1 : RFC 2068



Le protocole HTTP

HTTP :

- Le client initie une connexion TCP (crée une socket) avec le serveur, port 80
- Le serveur accepte la connexion TCP du client
- Les messages HTTP (protocole applicatif) sont échangés entre le browser (client HTTP) et le serveur Web
- La connexion TCP est close

HTTP est « sans état »

- Le serveur ne maintient aucune information au sujet des requêtes précédentes des clients

Les protocoles gardant un "état" sont complexes !

- ☐ L'histoire passée doit être gardée
- ☐ Si le serveur ou le client se crashe les états peuvent être restitués (pb d'incohérence)

Exemple HTTP

Si un utilisateur entre l'URL :

<http://www.someSchool.edu/someDepartment/home.index>

1a. Le client HTTP initie une connexion TCP au serveur HTTP sur le site www.someSchool.edu. Le port 80 est choisi par défaut

1b. Le serveur HTTP du site www.someSchool.edu attend une connexion TCP sur le port 80. Il "accepte" la connexion, et l'annonce au client

2. Le client HTTP envoie les *requêtes HTTP* (contenant des URLs) par les sockets TCP

3. Le serveur HTTP reçoit le message de requête, génère le *message de réponse* contenant l'objet requis (*someDepartment/home.index*), et l'envoie sur une socket

time



Exemple HTTP (suite)

4. Le serveur HTTP ferme la connexion TCP

5. Le client HTTP reçoit la réponse contenant le fichier HTML file et l'affiche.

En décodant le fichier, le browser trouve les URLs référencées

6. Les étapes 1-5 sont répétées pour chaque URL référencée

time



Fonctionnement des Prog. réseau : le protocole HTTP

Les requêtes clients Http



Format de message http : requête

☰ Deux types de messages http : *requête, réponse*

☰ message de requête http :

Ligne de requête
(commandes
GET, POST, HEAD) → GET /somedir/page.html HTTP/1.0

Lignes
d'entête [Host: www.someschool.edu
Connection: close
User-agent: Mozilla/4.0
Accept: text/html, image/gif, image/jpeg
Accept-language: fr

Le retour chariot
indique la fin
du message



Les requêtes clients Http

❑ La syntaxe de la demande (= requête client) est toujours la même :

Ligne de commande (**Commande**, URL, Version de protocole)
En-tête de requête
<nouvelle ligne>
Corps de requête

❑ Ligne de commande

Commande est la méthode à utiliser, il spécifie le type de requête, il peut avoir les valeurs :

✓GET

C'est la méthode la plus courante pour demander une ressource. Une requête **GET** est sans effet sur la ressource, il doit être possible de répéter la requête sans effet.

Les requêtes clients Http

✓ HEAD

Cette méthode ne demande que des informations sur la ressource, sans demander la ressource elle-même.

✓ POST

Cette méthode est utilisée généralement lorsqu'une requête ajoute une ressource dans le serveur.

✓ OPTIONS

Cette méthode permet d'obtenir les options de communication d'une ressource ou du serveur en général.

✓ CONNECT

Cette méthode permet d'utiliser un proxy comme un tunnel de communication.

Les requêtes clients Http

✓TRACE

Cette méthode demande au serveur de retourner ce qu'il a reçu, dans le but de tester et d'effectuer un diagnostic sur la connexion.

✓DELETE

Cette méthode permet de supprimer une ressource du serveur.

NB: les plus courantes (utilisés): **HEAD**, **GET** et **POST**.

Les requêtes clients Http

Ligne de commande (Commande, **URL**, **Version de protocole**)

En-tête de requête

<nouvelle ligne>

Corps de requête

✓ Ensuite, vient URL, c'est-à-dire l'adresse de la page sur le serveur elle est du type : "/repertoire/page.ext".

Traduction : le fichier "page.ext" dans le dossier "repertoire" qui se situe à la racine du serveur.

✓ Le dernier élément de Ligne de commande est la **version** de HTTP utilisée.

✓ La version utilisée est HTTP 1.1, on doit donc écrire : "HTTP/1.1".

Exemple:

```
GET /repertoire/page.html HTTP/1.1
```


Les requêtes clients Http

✓ En-têtes

celui-ci peut contenir tout un tas de valeurs (c'est un peu le même principe que les balises <meta> en HTML), toujours présentées sous la forme :

Nom: valeur

✓ Host: www.site.com

Spécifie le nom de domaine du site.

✓ Cookie: nom-cookie=valeur cookie

Permet d'envoyer au serveur les cookies qui ont été enregistrés.

Un cookie toujours un nom et une valeur. Pour en envoyer plusieurs, écrivez-les à la suite, séparés par un point-virgule.

Les requêtes clients Http

✓ **Connection: Close | Keep-Alive**

Définit le type de connexion :

- ✓ Close : la connexion est fermée après la réponse
- ✓ Keep-Alive : crée une connexion persistante. Avec ce type de connexion, il est même possible d'envoyer une requête sans attendre la réponse à la précédente.

✓ **Content-type: application/x-www-form-urlencoded**

les valeurs sont encodées sous forme de couples clé-valeur séparés par '&', avec un '=' entre la clé et la valeur.

```
MyVariableOne=ValueOne&MyVariableTwo=ValueTwo
```

✓ **Content-Length: 42**

Spécifie la longueur du corps de requête : la taille en octets.

```
Content-Length: 13  
say=Hi&to=Mom
```

Les requêtes clients Http

✓ Corps

Pour terminer la requête, on envoie le corps de requête. Il peut contenir, par exemple, le contenu d'un formulaire HTML envoyé en POST.

Dans le cas du formulaire, les variables ont un nom et une valeur, comme l'en-tête, Cookie, et les différentes variables sont séparées par desesperluettes : "&"

Exemple :

```
POST / HTTP/1.1
Host: foo.com
Content-Type: application/x-www-form-urlencoded
Content-Length: 13
say=Hi&to=Mom
```

Les requêtes clients Http

✓ GET

GET est la méthode permettant de récupérer le contenu d'un fichier.

Exemple

```
GET /fichier.ext HTTP/1.1  
Host: www.site.com  
Connection: Close
```

✓ Dans le PHP, **GET** est très utilisé: En effet, pour pouvoir récupérer des variables passées par l'url, il faut faire : `$_GET['variable']`.

Pour transmettre ces variables, il faut remplacer la ligne de commande par quelque chose du type :

```
GET /fichier.ext?variable=valeur&variable2=valeur2 HTTP/1.1
```

Les requêtes clients Http

✓ HEAD

HEAD demande l'envoi de l'en-tête de la réponse sans son contenu. Il est intéressant si on veut, par exemple, récupérer des informations sur un fichier ou sur le serveur.

La méthode HTTP HEAD demande les en-têtes qui seraient retournés si la ressource spécifiée était demandée avec une méthode HTTP GET.

Une telle requête peut être envoyée avant de procéder au téléchargement d'une ressource volumineuse, par exemple pour économiser de la bande passante.

Exemple :

```
HEAD /fichier.ext HTTP/1.1  
Host: www.site.com  
Connection: Close  
<nouvelle ligne>
```

Les requêtes clients Http

✓ POST

Ce dernier est un peu plus complexe. Il a le même effet que GET sauf que, lui, envoie des données via le corps de la requête.

Il faut spécifier le type des données. Pour un formulaire HTML, il y a 2 valeurs possibles :

- application/x-www-form-urlencoded
valeur par défaut

- multipart/form-data

utilisée notamment pour l'upload de fichiers dans le formulaire.

Il faut également spécifier la longueur du corps.

Exemple : POST /fichier.ext HTTP/1.1
Host: www.site.com
Connection: Close
Content-type: application/x-www-form-urlencoded
Content-Length: 33

variable=valeur&variable2=valeur2

Fonctionnement des Prog. réseau : le protocole HTTP

Les réponses Http



Les réponses Http

✓ Ligne de statut

La ligne de statut est composée de :

- Version : la version HTTP du serveur
- Code-réponse : le code d'erreur retourné (par exemple 200, 403, 404, 500)
- Texte-réponse : le texte associé à l'erreur (respectivement pour les exemples précédents : "OK", "FORBIDDEN", "NOT FOUND", "INTERNAL ERROR").

Ligne de statut (Version, Code-réponse, Texte-réponse)

En-tête de réponse

<nouvelle ligne>

Corps de réponse

Code	Message	Signification
200	<i>OK</i>	Requête traitée avec succès.
201	<i>Created</i>	Requête traitée avec succès et création d'un document.
403	<i>Forbidden</i>	Le serveur a compris la requête, mais refuse de l'exécuter.

Code de réponse HTTP

Dans la première ligne de la réponse serveur->client.

200 OK

- La requête a réussi et l'objet demandé est à la suite

301 Moved Permanently

- L'objet demandé a changé définitivement de place, son nouvel emplacement est donné dans la suite du message

400 Bad Request

- La requête est erronée

404 Not Found

- Le document demandé n'est pas disponible sur le serveur

505 HTTP Version Not Supported



Code de réponse HTTP

Dans la première ligne de la réponse serveur->client.

200 OK

- La requête a réussi et l'objet demandé est à la suite

301 Moved Permanently

- L'objet demandé a changé définitivement de place, son nouvel emplacement est donné dans la suite du message

400 Bad Request

- La requête est erronée

404 Not Found

- Le document demandé n'est pas disponible sur le serveur

505 HTTP Version Not Supported



Les réponses Http

En-têtes

Les en-têtes sont nombreux ici aussi, en voici quelques-uns utiles :

✓ Date: Fri, 31 Dec 1999 23:59:59 GMT

Date de génération de la réponse.

✓ Server: Apache/2.2.3

Spécifie le modèle du serveur HTTP.

✓ Content-type: text/plain

pour que la réponse s'affiche comme du texte brut

✓ Content-Length: 42

✓ Set-Cookie: variable=valeur; expires=Fri, 31-Dec-2010 23:59:59 GMT;
path=/; domain=.site.com

Càd. Demande au navigateur d'enregistrer un cookie (voir setcookie())

Format de message http : réponse

Ligne d'état
(protocole,
code d'état,
message d'état)

Lignes
d'entête

données, e.g.
Le fichier
html

```
HTTP/1.0 200 OK
Connection: close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Content-Length: 6821
Content-Type: text/html
...
data data data data data ...
```



Les réponses Http

✓ Corps

Le Corps de réponse contient le contenu du fichier, le HTML d'une page par exemple.

Exemple de réponse : Citation : www.siteduzero.com

```
HTTP/1.1 200 OK
```

```
Date: Thu, 11 Jan 2007 14:00:36 GMT
```

```
Server: Apache/2.0.54 (Debian GNU/Linux)
```

```
Connection: close
```

```
Transfer-Encoding: chunked
```

```
Content-Type: text/html; charset=ISO-8859-1178a1
```

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
```

```
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd">
```

```
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="fr">
```

```
<head>
```

```
<meta http-equiv="content-type" content="text/html; charset=ISO-8859-1" />
```

```
<meta http-equiv="pragma" content="no-cache" />
```

```
<title>Accueil - Le Site du Zéro</title>
```

```
...
```